

数据结构 考试范围

一 绪论

概念：数据、数据元素、数据结构、运算复杂度

- **数据** 客观事物的符号表示
 - 数值型数据 包括整数、实数、复数等
 - 非数值型数据 包括文字、图像、语音等
- **数据元素** 议案在计算机中作为整体进行处理，是数据的基本单位，数据元素也被称为**记录**；有的数据元素由若干**数据项**组成，就好像结构体的属性
- **数结构** 相互间存在一定关系的数据元素的集合
 - **逻辑结构图**可以表示数据关系
 - **逻辑结构**分为**线性结构**、**树状结构**和**图状结构**
 - 数据结构在计算机中的表示称为**物理结构**或**存储结构**
- **运算复杂度**
 - **时间复杂度 (Time Complexity)**：描述算法执行所需时间与输入数据规模之间的关系。常见的时间复杂度表示方法有：
 - **$O(1)$** ：常数时间，算法的执行时间不随输入数据规模变化。
 - **$O(\log n)$** ：对数时间，算法的执行时间随输入数据规模的对数变化。
 - **$O(n)$** ：线性时间，执行时间与输入数据规模成正比。
 - **$O(n \log n)$** ：线性对数时间，常见于分治法的算法（如归并排序、快速排序）。
 - **$O(n^2)$** ：平方时间，通常出现在双重循环中（如冒泡排序）。
 - **$O(2^n)$** ：指数时间，通常出现在暴力解法中。
 - **$O(n!)$** ：阶乘时间，常见于全排列类问题。
 - **空间复杂度 (Space Complexity)**：描述算法执行时所需要的空间（内存）与输入数据规模之间的关系。空间复杂度考虑了程序执行过程中临时使用的存储空间，尤其是递归和动态数据结构对空间的需求。

二 线性表

概念：线性表、顺序结构和链式结构

掌握：链表应用；空头结点；集合，合并，倒置等

- **线性表** 数据之间存在一对一线性关系，除第一个元素和最后一个元素外，每个元素都有一个前驱和后继
- **顺序结构** 存储地址连续 数组实现
- **链式结构** 存储地址不连续 链表实现

链表应用

- 内存管理 跟踪闲置内存块
- 实现其他数据结构 栈、队列和哈希表
- 特殊的链表
 - **循环链表** 最后一个节点指向头结点，无尾指针；用于轮询调度(循环)
 - **双向链表** 每个结点都能指向前驱后继，可以双向搜索

空头结点

空头结点用于简化链表操作，尤其是插入和删除操作

- 简化边界判断 执行插入头结点 `insertAtHead()` 过程不需要判空

```
void insertAtHead(Node*& head, int value) {
    Node* newNode = new Node{value, nullptr};
    newNode->next = head->next;
    head->next = newNode;
}
```

- 统一插入删除操作 通过结点变动实现，即令当前节点为 `current`

特别要注意 `current` 应用！

- 避免头指针处理 在删除头结点时无特殊考虑，只删除头后面的结点

线性表操作

- 集合
 - 并集 遍历线性表，获取所有不重复元素
 - 交集 遍历其中一个线性表，若其中元素存在于另一个线性表中，添加
 - 差集 遍历其中一个线性表，若其中元素不在另一个线性表中，添加
 - 合并 用两指针分别指向两线性表头部，若某指针指向元素较小，添加到新表中，并指针后移
- 合并前，两线性表已经有序
- 倒置 用两个指针，从头尾开始交换元素并移动(可随机访问的线性表)；或对链表，改变每个指针指向，指向前驱

三 栈和队列

概念：栈、队列、顺序结构和链式结构、循环队列

掌握：进退栈顺序

- 栈
 - 顺序栈 固定大小的栈，初始化 `top = -1`，判满为 `top == capacity - 1`

- 链式栈 维护一个 `top` 指针，初始时指向 `nullptr`，每次更新到链表尾部
- 队列
 - 顺序队列(循环队列) 维护 `front` 和 `rear` 两个指针，初始化为0，用取余操作实现循环队列
 - 入队 `rear = (rear + 1) % capacity`
 - 出队 `front = (front + 1) % capacity`
 - 判空 头尾指针相遇 `front == rear`
 - 判满 头尾指针紧邻，且头指针在尾指针前 `(rear + 1) % capacity == front`
 - 栈式队列 维护头尾指针 `front` 和 `rear`，创建和删除结点都对它们进行更新

栈的应用

- 用栈对前中后缀表达式进行转换 [2024年11月13日](#)
- 判断出栈队列的可能性
 1. 模拟操入栈操作：若当前出栈元素不在栈顶，不断从入栈序列中取元素入栈，直到找到该元素
 2. 若当前出栈元素在栈顶，弹出栈顶元素
 3. 若遍历完出栈序列，所有操作能够做完，则是，否则不是(此时模拟栈内存在元素，非空)

四 串

概念：串的基本概念

掌握：子串、主串

- 串 零或多个字符组成的有限序列，是线性结构。常见操作求长度、链接、比较(按照字典序)、子串提取、模式匹配

主串和子串

- 字符串匹配算法 暴力法 $O(n * m)$ 和KMP算法 $O(n + m)$
- 前后缀、最长公共子串、回文(双指针法 $O(n)$ ，位操作 $O(\log n)$)

五 数组和广义表

概念：数组、广义表定义和一般应用

掌握：矩阵元素的定位

- 数组：在连续内存地址中存储相同类型数据元素的数据结构
- 广义表：递归定义的数据结构；存储两种数据，子广义表和原子

表头是表的第一个元素，可以是广义表，也可以是原子；表尾一定是广义表(返回一个子表)

数组定位问题

注意：

- 数组维度大小 $x_{max}, y_{max}, z_{max}, \dots$
- 每个元素占位长度；元素的地址即为**首字节位置地址**
- 行列优先
 - 行优先：优先填满每行元素
 - 列优先：优先填满每列元素
- 对于坐标 (i, j) ，正规做法是求和法；选择题做法是代入验证法

六 树和二叉树

概念：树，二叉树，满二叉树，完全二叉树，树和森林

掌握：二叉树性质；二叉树存储结构；二叉树遍历；哈夫曼树

- 部分概念参考 笔记[笔记 第六章](#)
- **树和森林 考察孩子兄弟表示法**
 - 森林转树
 1. 对于森林中的每颗子树，进行孩子兄弟表示法转换。即第一个子节点转换为左孩子，右孩子表示其兄弟(在第一步中用不到，留给其他树)；剩余的子节点，表示为第一个子节点的右孩子(兄弟)
 2. 对于每颗子树，按顺序成为第一颗子树的兄弟；连接完成后，第一颗子树就是新二叉树的树根
 - 树转森林
 1. 树根的左孩子表示为树分裂后第一颗子树的孩子，根据孩子兄弟表示法形成子树
 2. 树根的右孩子表示为树分裂后其他子树的树根，这些右孩子的左孩子为新树根的子树

本章不考线索化二叉树

二叉树的性质

参考 [2024年12月13日](#)

注意：

- 小题尽量用验算法
- 完全二叉树节点标号注意开始是0还是1；用举例法获取规律
- 强化概念与编程关系，例如叶子结点必须是左右子节点指针为 `nullptr`

二叉树的存储结构

- **数组存储** 一般用于完全二叉树(防止稀疏树浪费空间)，关注索引和子树的关系
- **链式存储** 常用方案，包含左孩子和右孩子指针

- 其他存储结构有多叉树、线索二叉树等

遍历序列与树

- 有序遍历 前序、中序和后序 可以用来构建树(递归法构建)
- 层次遍历 用队列实现，注意层次遍历的序号

哈夫曼树

二叉哈夫曼树：

- 建造过程 详见笔记[笔记 第六章](#)

带权路径长度是指所有叶子结点权值与其到根节点路径长度的乘积和

非二叉哈夫曼树：为了减小树深度，或应对非二进制符号集(三进制、四进制等)，可以考虑 k 叉哈夫曼树。这种哈夫曼树每次从集合中获取 k 个结点，求和得到根节点并放回集合中，重复步骤直到集合中只剩一个节点可以得到

七 图

概念：图的各种定义、图的存储表示

掌握：图的遍历；最小生成树；拓扑排序和关键路径；最短路径问题

- 图的各种定义 参考[笔记](#)
- 图的存储表示
 - **邻接矩阵** 将所有的点之间存在路径的表示为1，否则表示为0；需要注意对角线为0或 ∞ ，即自己到自己的距离路径设置为0
 - **邻接表** 每个点后面接一个链表，存放能去的点的序号；要画成链表的形式

邻接矩阵的空间复杂度为 $O(n^2)$ ，邻接表的空间复杂度为 $O(n + e)$ ，其中 e 是边的数量

图的遍历

图的遍历分为：

- 深度优先遍历DLS 不到不能走不回溯
- 广度优先遍历BFS 按临近顺序遍历

这两种遍历分别可用**栈**和**队列**实现

最小生成树

用两种算法取得：

- Prim算法 一点出发加边得到
- Kruscal算法 放好全部点再加边得到

参考[2024年12月27日](#)

拓扑排序和关键路径

参考 [笔记](#)

最短路径问题

参考 [笔记](#)

八 查找

概念：查找的基本概念、静态查找表

掌握：二叉排序树；散列表

| 无AVL树

九 排序

概念：各种排序算法的步骤及时间复杂度

掌握：灵活运用排序算法构建解决新问题的方法

十 文件

概念：主存储器和辅存储器、顺序文件、索引文件、散列文件

- **主存** 直接与CPU交互的存储器，即RAM
- **辅存** 大容量、速度慢的硬盘
- **顺序文件** 直接按顺序依次存入文件
- **索引文件** 存储文件索引，再根据索引查找文件
- **散列文件** 存储文件散列后的哈希值，根据哈希值查找文件

总结

关于题型

- 重点章节：六、七、八、九
- 课本*章节不深入；无十一章

题型：

单选 30分 30小题

应用 48分 6题*8分

编程题 12分 1题

探索题 10分 1题

刷题经验

- 考察具体公式的小题，快速使用**验算公式法**